Assignment-3

Parallel & Distributed Computing (UCS645)

Submitted by:
Bhawesh War - 102313065

Submitted to:
Dr. Saif Nalband

Execution Time Analysis (1000 × 1000)

Mode / Threads	Execution Time (s)
Sequential (1 Thread)	1.44374
Parallel (1 Thread)	0.413643
Parallel (2 Threads)	0.307016
Parallel (4 Threads)	0.303388

Table 1: Execution Time Comparison

Observations:

- Parallel execution significantly reduces execution time compared to sequential execution.
- Even with 1 thread, parallel implementation performs better due to compiler optimizations and vectorization.
- Increasing threads from 1 to 2 improves performance noticeably.
- Increasing threads from 2 to 4 shows marginal improvement, indicating saturation.
- Performance is limited by hardware constraints and overhead.

Effect of Matrix Size (Sequential)

Matrix Size	Execution Time (s)
500 × 500	0.179375
1000 × 1000	1.42554
1500 × 1500	4.89434
2000 × 2000	11.4096

Table 2: Sequential Execution

Observations:

- Execution time increases rapidly with matrix size.
- Growth is non-linear due to increased computational complexity.
- Sequential execution becomes inefficient for large inputs.

Effect of Matrix Size (Parallel)

Threads = 1

Matrix Size	Execution Time (s)
500 × 500	0.0494332
1000 × 1000	0.376884
1500 × 1500	1.34676
2000 × 2000	3.12775

Table 3: Parallel (1 Thread)

Threads = 2

Matrix Size	Execution Time (s)
500 × 500	0.0432821
1000 × 1000	0.374461
1500 × 1500	1.00392
2000 × 2000	2.3726

Table 4: Parallel (2 Threads)

Threads = 4

Matrix Size	Execution Time (s)
500 × 500	0.0331478
1000 × 1000	0.276549
1500 × 1500	1.20579
2000 × 2000	2.20627

Table 5: Parallel (4 Threads)

Observations:

- Parallel execution consistently outperforms sequential execution.
- Increasing threads improves performance, especially for large matrices.
- Performance gain is limited beyond available CPU cores.
- Larger matrices benefit more from parallelism.
- Overhead and memory contention affect scaling at higher threads.

Perf Analysis

Matrix Size: 500 × 500

Metric	Sequential	Parallel (4 Threads)
Execution Time (s)	0.1873	0.0415
CPU Cycles	570,264,234	368,064,578
Instructions	396,150,834	432,873,303

Table 6: Perf Metrics (500 × 500)

Matrix Size: 1000 × 1000

Metric	Sequential	Parallel (4 Threads)
Execution Time (s)	1.4560	0.2805
CPU Cycles	4,458,468,011	1,962,825,671
Instructions	3,018,469,761	3,266,823,117

Table 7: Perf Metrics (1000 × 1000)

Matrix Size: 1500 × 1500

Metric	Sequential	Parallel (4 Threads)
Execution Time (s)	4.8772	1.0967
CPU Cycles	14,997,322,746	6,764,597,623
Instructions	10,040,139,367	10,863,548,772

Table 8: Perf Metrics (1500 × 1500)

Matrix Size: 2000 × 2000

Metric	Sequential	Parallel (4 Threads)
Execution Time (s)	11.5257	2.2349
CPU Cycles	35,333,409,340	15,071,425,983
Instructions	23,619,151,899	25,562,677,521

Table 9: Perf Metrics (2000 × 2000)

Observations:

- Parallel execution significantly reduces execution time for all matrix sizes.
- CPU cycles are consistently lower in parallel execution, indicating faster computation.
- Instruction count remains similar or slightly higher, showing the same workload is executed more efficiently.
- Performance improvement becomes more prominent as matrix size increases.
- Parallel efficiency is limited by hardware constraints and memory bandwidth.

Conclusion

- Parallelization using OpenMP significantly improves performance.
- Vectorization and compiler optimizations contribute to additional speedup.
- Larger datasets benefit more from parallel execution.
- Performance scaling is limited by hardware constraints.
- Efficient use of CPU resources is achieved through multithreading.